UADE

TPO PROGRAMACIÓN III
INFORME FINAL - SEGUNDA ENTREGA

Propuesta seleccionada: Propuesta 1 — Red Social Universitaria para la Colaboración

Grupo Viedma:

Luz Barrientos - 1193268

German Rocha Sanchez - 1195326

Martina Córdoba - 1152500

Materia: Programación III

Profesor: Leon Maria Angela

Fecha de entrega: 6/11/2026

Repositorio del proyecto:
https://github.com/luzbarrientos/TPO-Viedma-Red-Social

ÍNDICE

TPO PROGRAMACIÓN III 1
ÍNDICE 2
1. INTRODUCCIÓN Y OBJETIVOS 4
2. TECNOLOGÍAS Y ARQUITECTURA 4
2.1. Herramientas utilizadas 4
2.2. Organización del proyecto 5
2.3. Evolución del desarrollo 5
3. ORGANIZACIÓN DE PUBLICACIONES — MERGE SORT 6
3.1. Problema y restricciones 6
3.2. Estrategia y funcionamiento 6
3.3. Justificación y alternativas 7
3.4. Complejidad 7
3.5. Implementación y resultados 7
3.6. Pruebas 9
4. CAMINO DE DIFUSIÓN MÁS RÁPIDO — DIJKSTRA 9
4.1. Problema y restricciones 9
4.2. Estrategia y funcionamiento 9
4.3. Justificación y alternativas 9
4.4. Complejidad 10
4.5. Implementación y resultados 10
4.6. Pruebas 11
5. SELECCIÓN DE INICIATIVAS — MOCHILA 0/1 11
5.1. Problema y restricciones 11
5.2. Estrategia y funcionamiento 11
5.3. Justificación y alternativas 12
5.4. Complejidad 12
5.5. Implementación y resultados 12
5.6. Pruebas 14
6. SELECCIÓN DE ACTIVIDADES — A3 14
6.1. Problema y restricciones 14
6.2. Estrategia y funcionamiento 14
6.3. Justificación y alternativas 15
6.4. Complejidad 15
6.5. Implementación y resultados 15
6.6. Interfaz gráfica 16
6.7. Pruebas 17
7. FORMACIÓN DE EQUIPOS — B1 18
7.1. Problema y restricciones 18
7.2. Estrategia y funcionamiento 18
7.3. Podas utilizadas 18
7.4. Justificación y alternativas 19
7.5. Complejidad 19
7.6. Implementación y resultados 19

7.7. Pruebas 20
8. CADENAS ALTERNATIVAS DE DIFUSIÓN — B3 21
8.1. Problema y restricciones 21
8.2. Estrategia y funcionamiento 21
8.3. Podas utilizadas 21
8.4. Justificación y alternativas 22
8.5. Complejidad 22
8.6. Implementación y resultados 22
8.7. Pruebas 23
9. INTEGRACIÓN E INTERFAZ GRÁFICA 24
10. ANÁLISIS COMPARATIVO DE ALGORITMOS 24
10.1. Estrategias implementadas 24
10.2. Comparación 25
11. PRUEBAS Y RESULTADOS 25
11.1. Plan de pruebas 25
11.2. Resultados automatizados 26
12. ORGANIZACIÓN DEL TRABAJO GRUPAL 29
13. DIFICULTADES Y MEJORAS 30
14. CONCLUSIÓN 31
15. REFERENCIAS 31

1. INTRODUCCIÓN Y OBJETIVOS

El Trabajo Práctico Obligatorio de Programación III consiste en desarrollar una aplicación Java que resuelve problemas característicos de una red social universitaria orientada a la colaboración entre estudiantes.

El objetivo principal es aplicar distintas estrategias de Diseño y Análisis de Algoritmos, justificando su selección, analizando su complejidad y verificando su funcionamiento mediante pruebas.

El sistema implementa seis funcionalidades:

1. Ordenamiento de publicaciones mediante Merge Sort.
2. Búsqueda del camino de difusión más rápido mediante Dijkstra.
3. Selección de iniciativas mediante Mochila 0/1.
4. Selección de actividades compatibles mediante Programación Dinámica.
5. Formación de equipos con restricciones mediante Backtracking.
6. Búsqueda de cadenas alternativas de difusión mediante Backtracking.

Las tres primeras corresponden a las funcionalidades obligatorias de la primera etapa. Para la segunda etapa se incorporaron A3, B1 y B3.

El proyecto utiliza las cuatro estrategias solicitadas: Divide y Conquista, Greedy, Programación Dinámica y Backtracking.

El alcance se concentra en los problemas algorítmicos. Los datos se representan mediante objetos Java y estructuras en memoria, sin implementar servicios web, autenticación ni persistencia en bases de datos.

Como resultado, se obtuvo una aplicación modular con interfaz gráfica Swing, seis funcionalidades integradas y 43 pruebas automatizadas.

2. TECNOLOGÍAS Y ARQUITECTURA

2.1. Herramientas utilizadas

Java 21: lenguaje utilizado para implementar los algoritmos, modelos y pruebas.

IntelliJ IDEA: entorno empleado para desarrollar, compilar y ejecutar el proyecto.

Java Swing: biblioteca utilizada para construir la interfaz gráfica de escritorio.

Git y GitHub: herramientas de control de versiones y colaboración grupal.

2.2. Organización del proyecto

El código se organiza en paquetes con responsabilidades diferenciadas:

Paquete	**Responsabilidad**
redsocial.modelo	Entidades y estructuras del sistema
redsocial.algoritmos	Implementación de los seis algoritmos
redsocial.datos	Conjuntos de datos de ejemplo
redsocial.pruebas	Casos de prueba automatizados
redsocial.interfaz	Ventana principal y componentes Swing

Las clases algorítmicas principales son:

- MergeSortPublicaciones.java
- Dijkstra.java
- Mochila.java
- SeleccionActividades.java
- FormacionEquipos.java
- CadenasDifusion.java

La separación permite ejecutar y probar los algoritmos independientemente de la interfaz gráfica.

2.3. Evolución del desarrollo

La primera entrega incluyó las tres funcionalidades obligatorias, ejecutables desde consola.

En la segunda entrega se incorporaron los tres algoritmos adicionales, nuevas validaciones, pruebas automatizadas y una interfaz gráfica con seis pestañas.

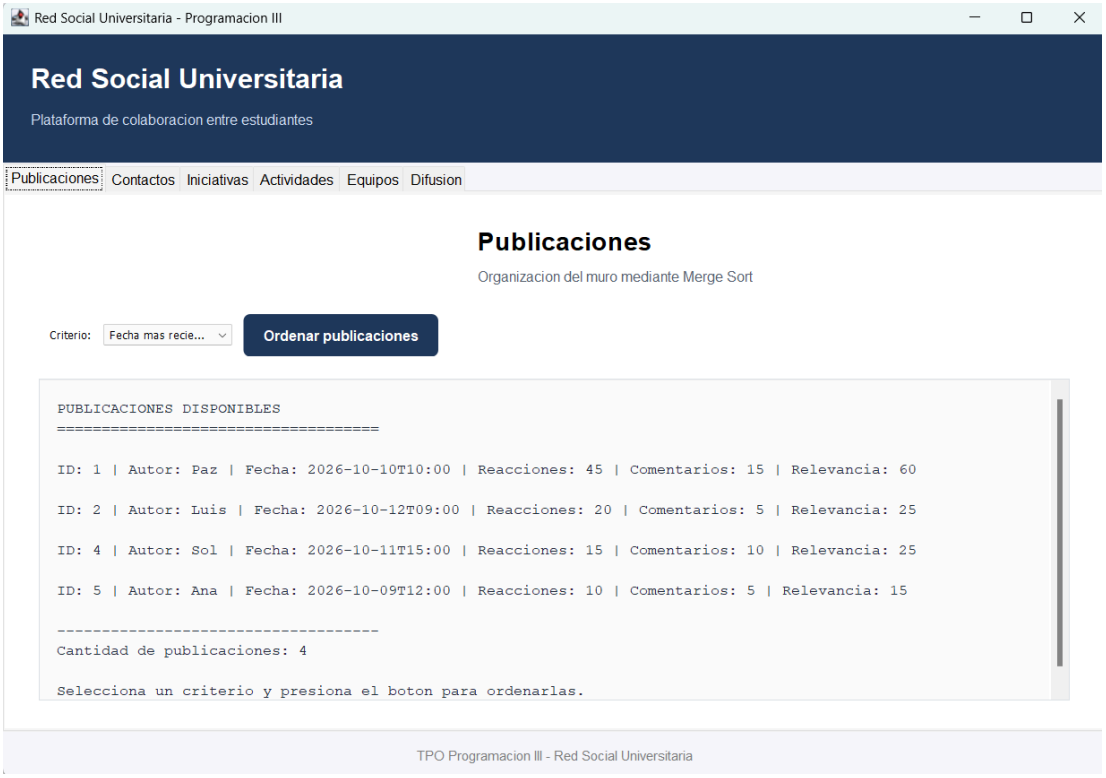


Figura 1. Ventana principal de la aplicación con las seis funcionalidades disponibles.

3. ORGANIZACIÓN DE PUBLICACIONES — MERGE SORT

3.1. Problema y restricciones

La red social debe organizar publicaciones según dos criterios:

- Fecha, desde la más reciente hasta la más antigua.
- Relevancia, desde la mayor hasta la menor.

Cada publicación posee identificador, autor, fecha, reacciones y comentarios.

La relevancia se calcula sumando reacciones y comentarios.

Para resolver empates se considera la fecha y posteriormente el identificador.

3.2. Estrategia y funcionamiento

Se implementó Merge Sort, basado en Divide y Conquista.

El algoritmo divide recursivamente el conjunto de publicaciones en dos mitades hasta obtener subconjuntos de un elemento. Luego combina las mitades respetando el criterio de ordenamiento.

Esta estrategia permite ordenar eficientemente sin depender del orden inicial de los datos.

3.3. Justificación y alternativas

Se eligió Merge Sort porque garantiza una complejidad temporal O(n log n) en todos los casos.

Se descartó Bubble Sort por su complejidad $O(n^2)$ en el peor caso. Quick Sort también fue considerado, pero puede alcanzar $O(n^2)$ en determinadas entradas.

3.4. Complejidad

Tiempo: O(n log n).
Espacio auxiliar: O(n).

Donde n representa la cantidad de publicaciones.

3.5. Implementación y resultados

Clase: MergeSortPublicaciones.java.

La interfaz permite elegir el criterio y ejecutar el ordenamiento.

Con los datos de ejemplo se obtuvo:

Orden por fecha	**Orden por relevancia**
Luis — 12/10	Paz — 60
Sol — 11/10	Luis — 25
Paz — 10/10	Sol — 25
Ana — 09/10	Ana — 15

Los resultados coinciden con los criterios establecidos.



Figura 2. Ordenamiento de publicaciones desde la más reciente.

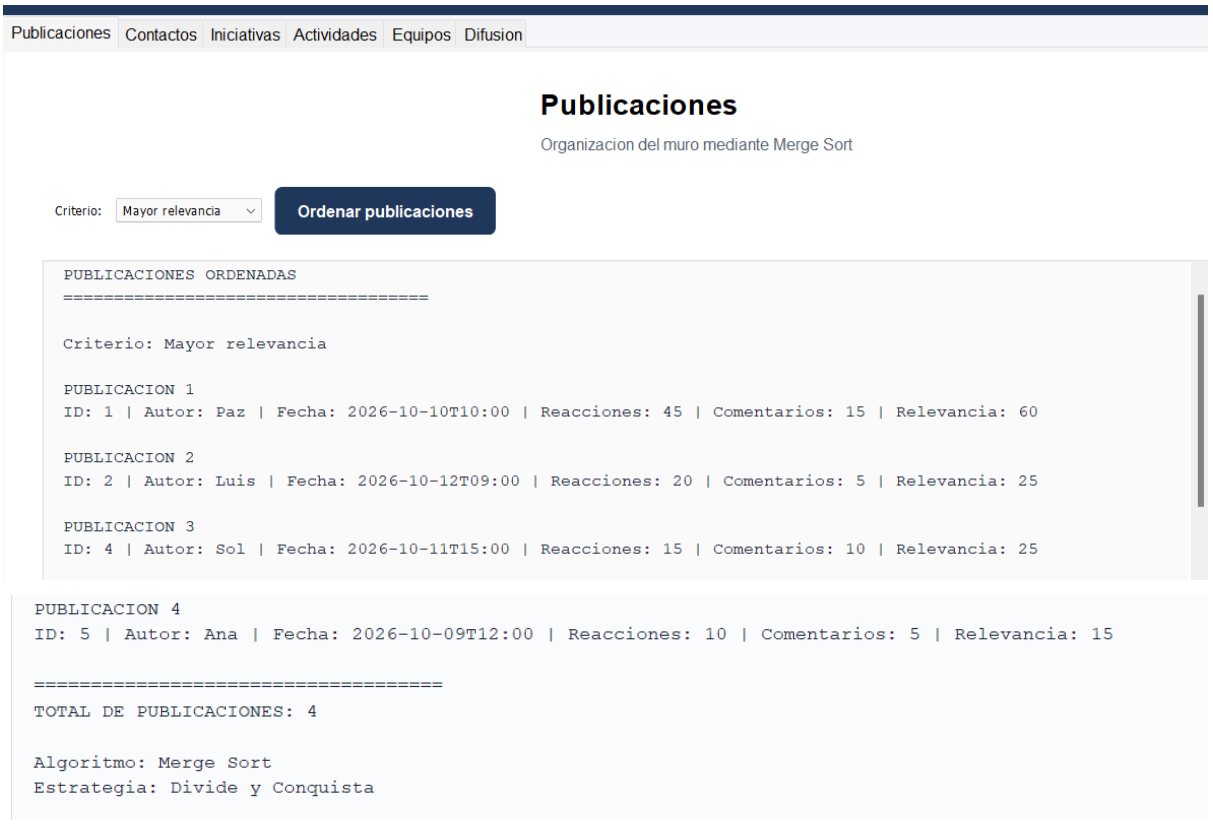


Figura 3. Ordenamiento de publicaciones por relevancia.

3.6. Pruebas

Se implementaron seis casos, CP01–CP06, que verifican los criterios de ordenamiento, empates, lista vacía, un único elemento y conservación del arreglo original.

Resultado: 6 pruebas correctas, 0 fallidas.

4. CAMINO DE DIFUSIÓN MÁS RÁPIDO — DIJKSTRA

4.1. Problema y restricciones

Se requiere encontrar la cadena de menor tiempo entre dos usuarios de la red social.

La red se representa mediante un grafo dirigido y ponderado:

- Los vértices representan usuarios.
- Las aristas representan vínculos de difusión.
- Los pesos representan tiempos de transmisión positivos.

El resultado debe informar el camino y su tiempo total, contemplando también destinos inalcanzables.

4.2. Estrategia y funcionamiento

Se implementó Dijkstra, basado en la estrategia Greedy.

El algoritmo comienza asignando distancia cero al origen e infinito a los demás usuarios.

Mediante una cola de prioridad selecciona el usuario pendiente con menor distancia conocida y actualiza las distancias de sus vecinos cuando encuentra recorridos más rápidos.

Los predecesores permiten reconstruir el camino obtenido.

4.3. Justificación y alternativas

Dijkstra resulta adecuado porque los pesos de la red son positivos.

Se descartó BFS porque minimiza la cantidad de aristas, no necesariamente el tiempo total cuando existen pesos diferentes.

También se consideró Floyd-Warshall, pero resuelve caminos mínimos entre todos los pares de vértices, mientras que el problema requiere una consulta entre origen y destino.

4.4. Complejidad

Tiempo: O((V + E) log V).
Espacio: O(V + E), incluyendo la representación de la red.

Donde V es la cantidad de usuarios y E la cantidad de vínculos.

4.5. Implementación y resultados

Clase: Dijkstra.java.

Con los datos de ejemplo:

Origen: Ana.
Destino: Eva.

Camino encontrado:

Ana → Caro → Beto → Dani → Eva.

Tiempo mínimo: 11 minutos.
Usuarios en la cadena: 5.

El resultado coincide con el esperado para la red de ejemplo.

Ejemplo de ejecución del algoritmo

Para comprender el funcionamiento de Dijkstra, se considera la búsqueda del camino más rápido desde Ana hasta Eva.

El algoritmo comienza asignando una distancia de 0 a Ana y una distancia infinita a los demás usuarios. A partir de ese momento, selecciona sucesivamente el usuario con menor distancia conocida y actualiza los tiempos de sus vecinos.

El procedimiento continúa hasta determinar la distancia mínima al destino. Durante la ejecución, se almacenan los predecesores de cada usuario para reconstruir el recorrido final.

En el ejemplo utilizado, la cadena óptima es: Ana → Caro → Beto → Dani → Eva

Tiempo total mínimo: 11 minutos.

Este resultado demuestra que Dijkstra no selecciona necesariamente el recorrido con menor cantidad de usuarios, sino aquel cuya suma de tiempos es mínima.

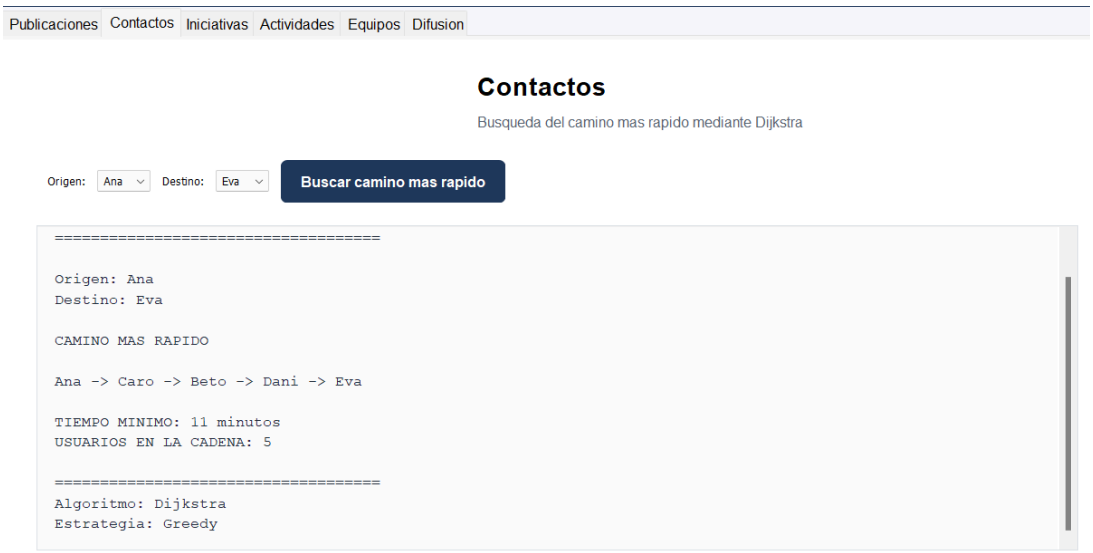


Figura 4. Camino mínimo entre Ana y Eva, con un tiempo de 11 minutos.

4.6. Pruebas

Se implementaron seis casos, CP13–CP18, que incluyen camino mínimo, destino inalcanzable, origen igual a destino, usuario inexistente, camino indirecto y registro de decisiones.

Resultado: 6 pruebas correctas, 0 fallidas.

5. SELECCIÓN DE INICIATIVAS — MOCHILA 0/1

5.1. Problema y restricciones

La universidad dispone de un presupuesto limitado para financiar iniciativas.

Cada iniciativa tiene un costo y un puntaje de impacto.

Se busca seleccionar la combinación que maximice el impacto total sin superar el presupuesto.

Cada iniciativa puede seleccionarse una sola vez y debe financiarse completamente.

5.2. Estrategia y funcionamiento

Se implementó Mochila 0/1 mediante Programación Dinámica.

Para cada iniciativa y presupuesto disponible se comparan dos posibilidades:

- No seleccionar la iniciativa.
- Seleccionarla, si el presupuesto alcanza.

Se conserva la alternativa con mayor impacto acumulado.

Los resultados parciales se almacenan para evitar cálculos repetidos.

5.3. Justificación y alternativas

La Programación Dinámica permite obtener una solución óptima.

Se descartó una estrategia Greedy porque elegir iniciativas por impacto individual o relación impacto/costo no garantiza la mejor combinación.

La fuerza bruta también permitiría resolver el problema, pero requiere explorar hasta 2^n subconjuntos.

5.4. Complejidad

Tiempo: O(n × P).
Espacio: O(n × P).

Donde n representa las iniciativas y P el presupuesto.

La complejidad es pseudopolinomial porque depende del valor numérico del presupuesto.

5.5. Implementación y resultados

Clase: Mochila.java.

Con un presupuesto de 10 se obtuvo:

Iniciativa	Costo	Impacto
2	5	9
3	3	6
4	2	4
Total	10	19

Impacto máximo: 19.
Presupuesto restante: 0.

La combinación respeta el límite presupuestario.

Ejemplo de ejecución mediante Programación Dinámica

Para ver el funcionamiento del algoritmo, se considera un presupuesto disponible de 10 unidades.

En cada etapa, el algoritmo evalúa si conviene incluir o excluir una iniciativa, teniendo en cuenta su costo, su impacto y el presupuesto restante.

La decisión se obtiene mediante la siguiente recurrencia:

DP(i, p) = max(DP(i−1, p), impacto(i) + DP(i−1, p−costo(i)))

La segunda alternativa se considera únicamente cuando el costo de la iniciativa no supera el presupuesto disponible.

Una vez completada la tabla de Programación Dinámica, se reconstruye la selección óptima.

Iniciativa seleccionada	Costo	Impacto	Presupuesto restante
2	5	9	5
3	3	6	2
4	2	4	0
Total	10	19	0

La combinación seleccionada utiliza todo el presupuesto disponible y alcanza un impacto total de 19.

El procedimiento permite evaluar distintas combinaciones y conservar aquella que proporciona el mayor impacto sin superar el presupuesto establecido.



```
Iniciativa 3:
Iniciativa 4 | Costo: 2 | Impacto: 4

==================================
CANTIDAD SELECCIONADA: 3
COSTO TOTAL: 10
IMPACTO MAXIMO: 19
PRESUPUESTO RESTANTE: 0

Algoritmo: Mochila 0/1
Estrategia: Programacion Dinamica
```

Figura 5. Selección óptima de iniciativas con presupuesto 10 e impacto 19.

5.6. Pruebas

Se implementaron seis casos, CP07–CP12, que contemplan diferentes presupuestos, presupuesto cero, lista vacía y entradas inválidas.

Resultado: 6 pruebas correctas, 0 fallidas.

6. SELECCIÓN DE ACTIVIDADES — A3

6.1. Problema y restricciones

La universidad organiza talleres y charlas que poseen horarios de inicio, finalización y un puntaje de interés.

Se requiere seleccionar actividades compatibles que maximicen el interés total.

Las restricciones son:

- Las actividades seleccionadas no pueden superponerse.
- Una actividad puede comenzar cuando termina otra.
- Cada actividad puede seleccionarse como máximo una vez.
- Se busca maximizar el interés, no necesariamente la cantidad de actividades.

6.2. Estrategia y funcionamiento

Se utilizó Programación Dinámica mediante selección ponderada de intervalos.

El algoritmo realiza los siguientes pasos:

1. Ordena las actividades por horario de finalización.
2. Determina para cada actividad la última actividad anterior compatible, utilizando búsqueda binaria.
3. Calcula el interés máximo comparando incluir o excluir cada actividad.
4. Reconstruye el conjunto de actividades seleccionado.

La recurrencia es: OPT(i) = max(OPT(i−1), interés(i) + OPT(p(i)))

Donde p(i) representa el índice de la última actividad anterior compatible con la actividad i. Si no existe una actividad compatible, se utiliza el caso base de la recurrencia.

6.3. Justificación y alternativas

Se eligió Programación Dinámica porque el problema requiere maximizar un valor acumulado y presenta subproblemas superpuestos.

La selección Greedy por horario de finalización permite maximizar la cantidad de actividades en el problema clásico, pero no garantiza el mayor interés cuando existen puntajes diferentes.

La fuerza bruta y Backtracking podrían explorar combinaciones, aunque con un costo potencialmente exponencial.

6.4. Complejidad

Tiempo: O(n log n).
Espacio auxiliar: O(n), sin incluir la memoria interna del ordenamiento.

El ordenamiento y las búsquedas binarias determinan el costo dominante.

6.5. Implementación y resultados

Clase: SeleccionActividades.java.

Se utilizaron las siguientes actividades:

Actividad	**Horario**	**Interés**
Taller Java	09:00–11:00	8
Taller Python	10:00–12:00	12
Charla IA	11:00–13:00	9
Taller Git	13:00–14:00	5

La selección óptima fue:

- Taller Java: 8.
- Charla IA: 9.
- Taller Git: 5.

Actividades seleccionadas: 3.
Interés máximo: 22.

Las actividades no se superponen y la solución coincide con el óptimo esperado.

Ejemplo de ejecución de la selección ponderada

Para mostrar el funcionamiento de la Programación Dinámica, se utilizan las cuatro actividades del conjunto de ejemplo.

Primero, las actividades se ordenan por horario de finalización. Luego, para cada actividad, se identifica la última actividad anterior compatible.

Actividad	Interés	Última compatible	Interés óptimo acumulado
Taller Java	8	Ninguna	8
Taller Python	12	Ninguna	12
Charla IA	9	Taller Java	17
Taller Git	5	Charla IA	22

Análisis de las decisiones:

1. Taller Java: se obtiene un interés acumulado de 8.
2. Taller Python: se compara su interés de 12 con el valor anterior de 8. Se conserva 12.
3. Charla IA: puede combinarse con Taller Java, alcanzando un interés de 17, superior al máximo anterior de 12.
4. Taller Git: puede combinarse con la selección que termina en Charla IA, alcanzando un interés total de 22.

Al reconstruir la solución, el algoritmo selecciona Taller Java, Charla IA y Taller Git.

Resultado final: 3 actividades compatibles con interés máximo de 22.

Este ejemplo demuestra por qué una selección basada únicamente en el interés individual no garantiza la solución óptima, mientras que la Programación Dinámica permite comparar combinaciones y maximizar el interés total.

6.6. Interfaz gráfica

La pestaña Actividades permite agregar, editar, eliminar y restaurar actividades, además de calcular la selección óptima.

Esto permite modificar los datos antes de ejecutar el algoritmo.



Figura 6. Selección óptima de tres actividades con interés máximo 22.



Figura 7. Controles de administración de actividades en Swing.

6.7. Pruebas

Se implementaron ocho casos, CP19–CP26, que verifican los datos de ejemplo, listas vacías, actividades únicas, superposiciones, horarios consecutivos, combinaciones óptimas, empates y horarios inválidos.

Resultado: 8 pruebas correctas, 0 fallidas.

7. FORMACIÓN DE EQUIPOS — B1

7.1. Problema y restricciones

La aplicación debe generar todos los equipos válidos de estudiantes respetando:

- Tamaño exacto del equipo.
- Habilidades requeridas.
- Integrantes obligatorios.
- Parejas incompatibles.
- Ausencia de equipos duplicados.

Cada estudiante posee un identificador y un conjunto de habilidades.

El objetivo es enumerar todas las combinaciones válidas.

7.2. Estrategia y funcionamiento

Se implementó Backtracking.

Para cada estudiante se consideran dos decisiones: incluirlo o excluirlo del equipo actual.

Cuando se incluye un estudiante, se verifica previamente su compatibilidad con los integrantes seleccionados.

Al alcanzar el tamaño solicitado, se comprueban las habilidades requeridas y los integrantes obligatorios.

Si el equipo es válido, se agrega a los resultados.

Posteriormente, el algoritmo deshace la decisión y continúa explorando otras combinaciones.

7.3. Podas utilizadas

La implementación evita continuar determinadas ramas cuando:

- Un estudiante es incompatible con el equipo actual.
- No quedan suficientes estudiantes para completar el equipo.
- Se alcanzó el tamaño solicitado.

La exploración por índices evita generar una misma combinación en diferentes órdenes.

7.4. Justificación y alternativas

Backtracking es adecuado porque se deben encontrar todas las soluciones válidas, no una única solución.

Una estrategia Greedy podría seleccionar un equipo, pero no garantizaría encontrar todas las combinaciones posibles.

La fuerza bruta sin poda exploraría combinaciones que podrían descartarse anticipadamente.

7.5. Complejidad

Tiempo: exponencial, con cota conservadora $O(n \times 2^n)$.
Espacio auxiliar: O(n), sin contar los resultados almacenados.

Donde n representa la cantidad de estudiantes.

Las podas reducen la exploración en muchos casos, aunque no eliminan el crecimiento exponencial del peor escenario.

7.6. Implementación y resultados

Clase: FormacionEquipos.java.

Con los datos de ejemplo se obtuvieron:

Equipos válidos: 4.

Los equipos poseen tres integrantes y cumplen las restricciones establecidas.

Entre los resultados aparece el equipo formado por Ana, Diego y Elena.

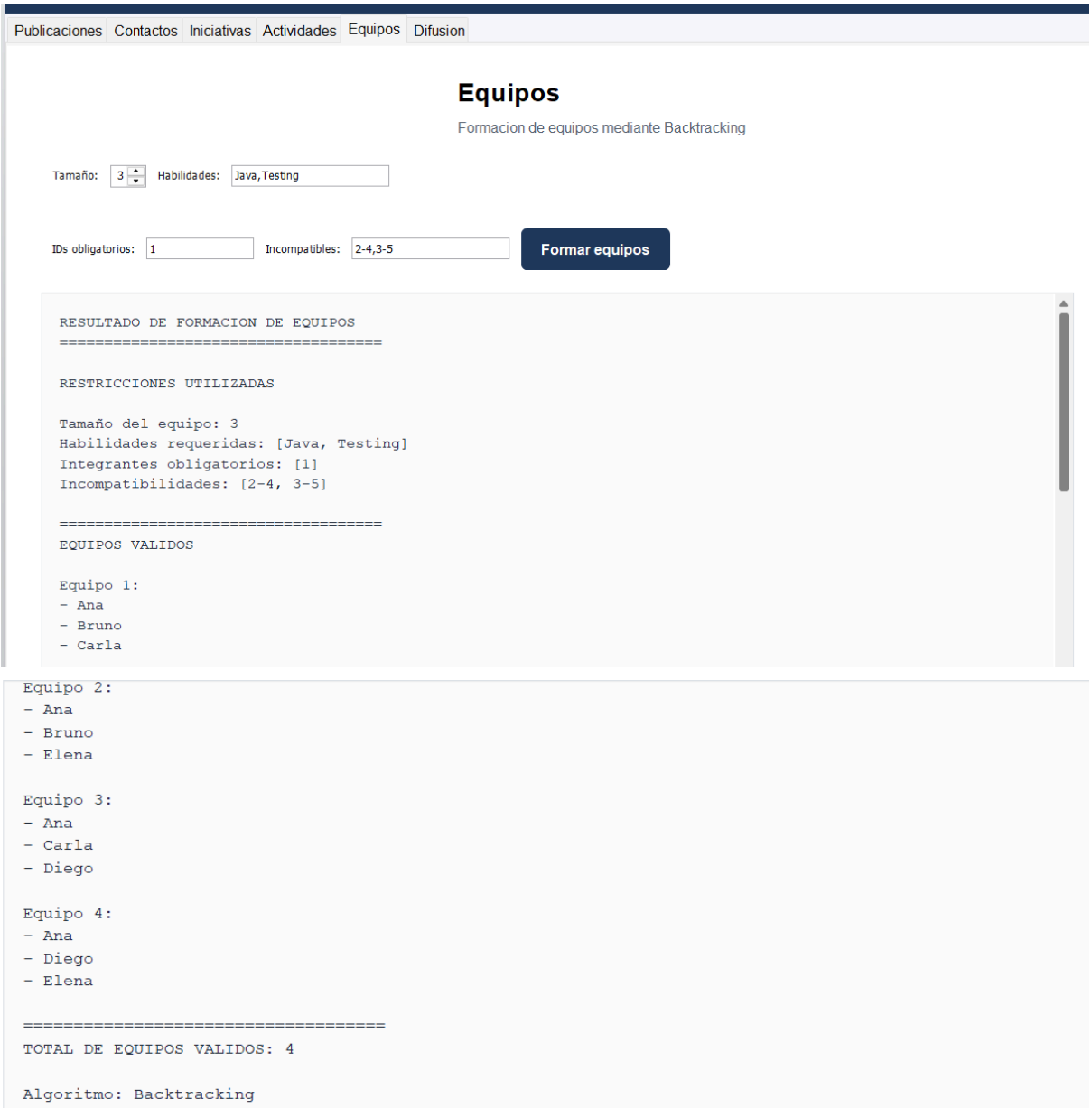


Figura 8. Equipos válidos obtenidos mediante Backtracking.

7.7. Pruebas

Se implementaron ocho casos, CP27–CP34, que verifican equipos esperados, tamaño, habilidades, integrantes obligatorios, incompatibilidades, ausencia de soluciones, duplicados y entradas inválidas.

Resultado: 8 pruebas correctas, 0 fallidas.

8. CADENAS ALTERNATIVAS DE DIFUSIÓN — B3

8.1. Problema y restricciones

La aplicación debe encontrar todas las cadenas válidas de difusión entre dos usuarios.

Cada vínculo posee un tiempo de transmisión.

Las restricciones incluyen:

- Tiempo máximo.
- Cantidad máxima de intermediarios.
- Usuarios excluidos.
- Prohibición de repetir usuarios en una cadena.

Además, se debe identificar la cadena más rápida entre las encontradas.

8.2. Estrategia y funcionamiento

Se implementó Backtracking sobre un grafo dirigido y ponderado.

El algoritmo comienza en el origen y explora los vínculos disponibles.

Durante la búsqueda mantiene la cadena actual, el conjunto de usuarios visitados y el tiempo acumulado.

Cuando alcanza el destino respetando las restricciones, registra la cadena como válida.

Después retrocede para explorar otras alternativas.

Al finalizar, identifica la cadena con menor tiempo total.

8.3. Podas utilizadas

Se descartan recorridos cuando:

- Superan el tiempo máximo.
- Exceden la cantidad permitida de intermediarios.
- Incluyen usuarios excluidos.
- Intentan repetir un usuario.

Estas restricciones reducen las ramas exploradas.

8.4. Justificación y alternativas

Backtracking permite enumerar todas las cadenas válidas.

Dijkstra no resulta suficiente porque encuentra un camino mínimo, pero no enumera todas las alternativas que cumplen restricciones.

BFS tampoco resuelve directamente este problema de enumeración con tiempos ponderados.

8.5. Complejidad

Tiempo: factorial en el peor caso, asociado a la cantidad de caminos simples posibles en redes densas.

Espacio auxiliar: O(V), sin contar el almacenamiento de las cadenas encontradas.

Donde V representa la cantidad de usuarios.

Los límites de tiempo e intermediarios permiten reducir la exploración en los casos prácticos.

8.6. Implementación y resultados

Clase: CadenasDifusion.java.

Parámetros utilizados:

Origen: Ana.
Destino: Eva.
Tiempo máximo: 16 minutos.
Intermediarios máximos: 3.
Usuarios excluidos: ninguno.

La aplicación encontró tres cadenas válidas.

La más rápida fue:

Ana → Caro → Beto → Dani → Eva.

Tiempo mínimo: 11 minutos.

Otra cadena observada fue Ana → Caro → Dani → Eva, con un tiempo de 15 minutos.

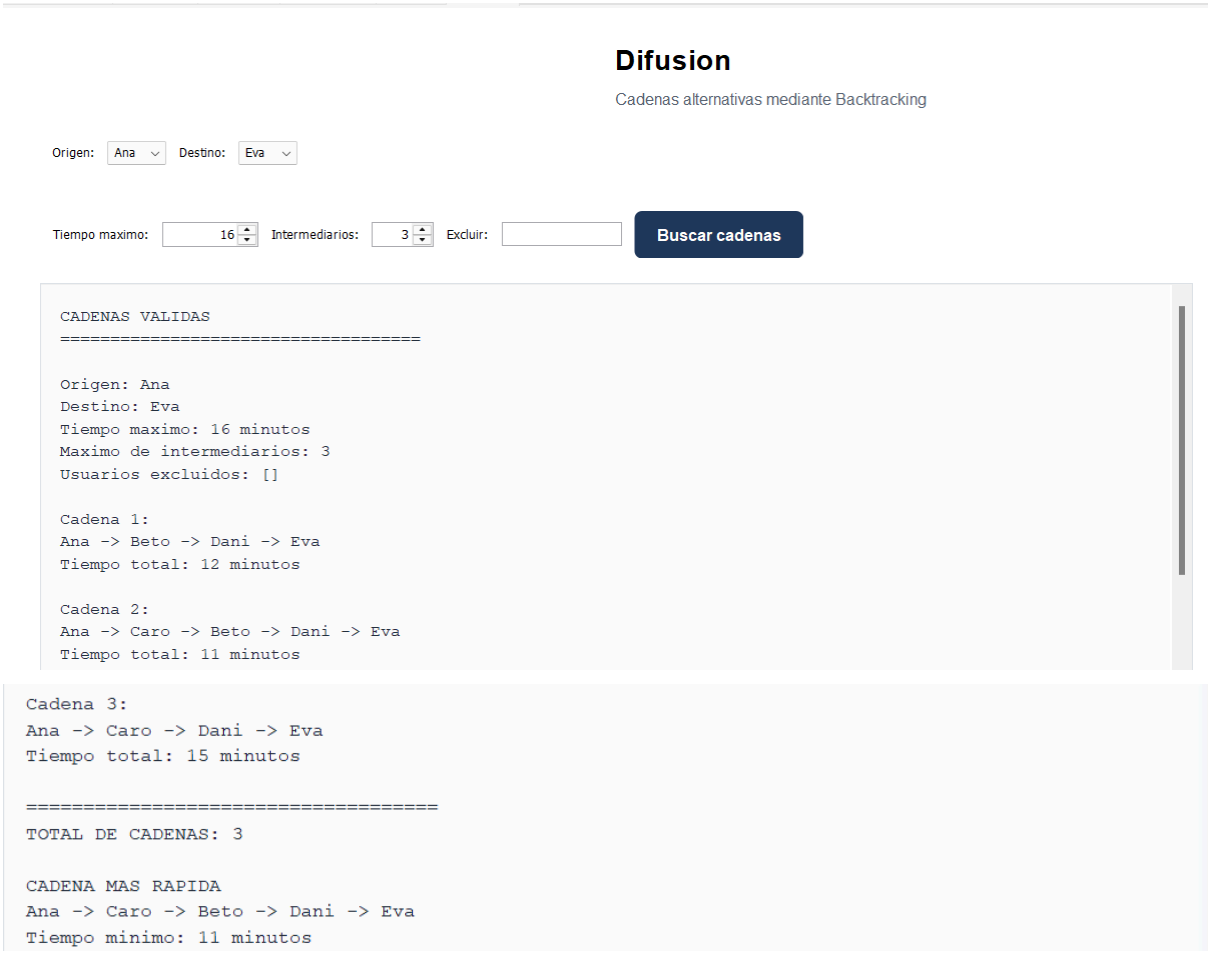


Figura 9. Tres cadenas válidas y cadena más rápida de 11 minutos.

8.7. Pruebas

Se implementaron nueve casos, CP35–CP43, que verifican cadenas esperadas, tiempo mínimo, límites, usuarios excluidos, ausencia de solución, usuarios repetidos, origen igual al destino y entradas inválidas.

Resultado: 9 pruebas correctas, 0 fallidas.

9. INTEGRACIÓN E INTERFAZ GRÁFICA

Para la entrega final se desarrolló una interfaz gráfica mediante Java Swing.

La ventana principal organiza las funcionalidades en seis pestañas:

Pestaña	**Operación**
Publicaciones	Ordenar por fecha o relevancia
Contactos	Buscar el camino más rápido
Iniciativas	Calcular la selección óptima
Actividades	Administrar y seleccionar actividades
Equipos	Generar equipos válidos
Difusión	Buscar cadenas alternativas

Las clases principales son:

- MainGUI.java
- VentanaPrincipal.java
- PanelFuncionalidades.java
- PanelFuncionalidadesExtra.java.

La interfaz recibe parámetros, ejecuta los algoritmos y muestra los resultados. La lógica algorítmica permanece separada, lo que facilita su reutilización y verificación.

Las seis funcionalidades se ejecutaron manualmente con los datos de ejemplo y mostraron los resultados esperados.

10. ANÁLISIS COMPARATIVO DE ALGORITMOS

10.1. Estrategias implementadas

Funcionalidad	**Estrategia**	**Complejidad temporal**
Merge Sort	Divide y Conquista	O(n log n)
Dijkstra	Greedy	O((V + E) log V)
Mochila 0/1	Programación Dinámica	O(n × P)
Actividades A3	Programación Dinámica	O(n log n)
Equipos B1	Backtracking	Exponencial
Difusión B3	Backtracking	Factorial en el peor caso

10.2. Comparación

Las estrategias se seleccionaron de acuerdo con los objetivos de cada problema.

Divide y Conquista permite resolver el ordenamiento mediante divisiones y combinaciones sucesivas.

Greedy resulta adecuado para Dijkstra porque los tiempos de los vínculos son positivos y permiten tomar decisiones óptimas de manera incremental.

Programación Dinámica permite resolver problemas de optimización reutilizando resultados de subproblemas.

Backtracking permite enumerar combinaciones o recorridos respetando restricciones y descartando ramas inválidas.

Las diferencias de complejidad muestran que no existe una única estrategia conveniente para todos los problemas.

En particular, las funcionalidades de Backtracking pueden presentar un costo elevado cuando aumenta el tamaño de la entrada, pero son adecuadas para la enumeración de soluciones requerida.

11. PRUEBAS Y RESULTADOS

11.1. Plan de pruebas

Se implementaron pruebas automatizadas para verificar el funcionamiento de los seis algoritmos.

Los escenarios contemplan resultados normales, casos límite, restricciones y entradas inválidas.

La clase EjecutarPruebas.java permite ejecutar los casos y mostrar un resumen general.

11.2. Resultados automatizados

Funcionalidad	Casos	Correctas	Fallidas
Merge Sort	CP01–CP06	6	0
Mochila 0/1	CP07–CP12	6	0
Dijkstra	CP13–CP18	6	0
Actividades A3	CP19–CP26	8	0
Equipos B1	CP27–CP34	8	0
Difusión B3	CP35–CP43	9	0
TOTAL	**43 casos**	**43**	**0**

La ejecución final documentada informó:

RESULTADO: 43/43 PRUEBAS CORRECTAS.

TODAS LAS FUNCIONALIDADES APROBADAS.

El proceso finalizó con código de salida 0.

11.3. Evidencias de pruebas automatizadas

```
---
 TPO PROGRAMACION III
 PRUEBAS DE LA PRIMERA ENTREGA
---

- PRUEBAS MERGE SORT -
[OK] CP01 - Ordenamiento por fecha
[OK] CP02 - Ordenamiento por relevancia
[OK] CP03 - Empate por relevancia y fecha
[OK] CP04 - Arreglo vacio
[OK] CP05 - Una sola publicacion
[OK] CP06 - No modifica el arreglo original
Correctas: 6 | Fallidas: 0

- PRUEBAS MOCHILA -
[OK] CP07 - Presupuesto 9
[OK] CP08 - Presupuesto 2
[OK] CP09 - Presupuesto 0
[OK] CP10 - Presupuesto suficiente
[OK] CP11 - Sin iniciativas
[OK] CP12 - Presupuesto negativo rechazado
Correctas: 6 | Fallidas: 0
```

```
- PRUEBAS DIJKSTRA -
[OK] CP13 - Camino minimo Ana a Eva
[OK] CP14 - Destino inalcanzable
[OK] CP15 - Origen igual a destino
[OK] CP16 - Usuario inexistente
[OK] CP17 - Camino indirecto mas rapido
[OK] CP18 - Registra decisiones
Correctas: 6 | Fallidas: 0

- RESUMEN GENERAL -
Merge Sort: CORRECTO
Mochila: CORRECTO
Dijkstra: CORRECTO

RESULTADO: 18/18 PRUEBAS CORRECTAS

Process finished with exit code 0
```

Figura 10.Resultado de las 18 pruebas automatizadas correspondientes a Merge Sort, Mochila y Dijkstra.

```
--- PRUEBAS SELECCION DE ACTIVIDADES ---
[Ok] CP19 - Datos de ejemplo
[Ok] CP20 - Lista vacia
[Ok] CP21 - Una actividad
[Ok] CP22 - Actividades superpuestas
[Ok] CP23 - Actividades consecutivas
[Ok] CP24 - Optimo combinado
[Ok] CP25 - Empate entre soluciones
[Ok] CP26 - Horario invalido
Correctas: 8 | Fallidas: 0

=== PRUEBAS B1: FORMACION DE EQUIPOS ===
[OK] CP27 - Equipos esperados
[OK] CP28 - Tamanio de equipo
[OK] CP29 - Habilidades requeridas
[OK] CP30 - Integrantes obligatorios
[OK] CP31 - Incompatibilidades
[OK] CP32 - Sin soluciones
[OK] CP33 - Sin equipos repetidos
[OK] CP34 - Entrada invalida

Correctas: 8
Fallidas: 0
```

Figura 11. Pruebas de selección de actividades y formación de equipos.



Figura 12. Pruebas de difusión y resumen general de 43 casos correctos.

También se ejecutaron las seis funcionalidades desde la interfaz gráfica.

Funcionalidad	**Resultado observado**
Publicaciones	Ordenamiento correcto por ambos criterios
Contactos	Camino mínimo de 11 minutos
Iniciativas	Impacto 19 con presupuesto 10
Actividades	Interés máximo 22
Equipos	Cuatro equipos válidos
Difusión	Tres cadenas; mínimo 11 minutos

Los resultados coinciden con los datos de ejemplo y complementan las pruebas automatizadas.

La documentación detallada de los casos se encuentra en el archivo pruebas/CASOS_DE_PRUEBA.

Los resultados obtenidos demuestran el funcionamiento esperado en los escenarios evaluados, sin implicar que se hayan probado todas las entradas posibles.

12. ORGANIZACIÓN DEL TRABAJO GRUPAL

El proyecto fue desarrollado por el Grupo Viedma, integrado por Luz Barrientos, Germán Rocha Sánchez y Martina Córdoba.

El trabajo se organizó en dos etapas:

- Durante la primera se implementaron los tres algoritmos obligatorios y las pruebas iniciales.
- Durante la segunda se incorporaron las funcionalidades A3, B1 y B3, se amplió la cobertura de pruebas, se agregaron validaciones y se desarrolló la interfaz gráfica.

Para coordinar el desarrollo se utilizaron Git y GitHub.

El repositorio compartido permitió registrar modificaciones mediante commits y mantener un historial del trabajo realizado.

También se actualizó el archivo README.md y se incorporó documentación de los 43 casos de prueba.

Repositorio: https://github.com/luzbarrientos/TPO-Viedma-Red-Social

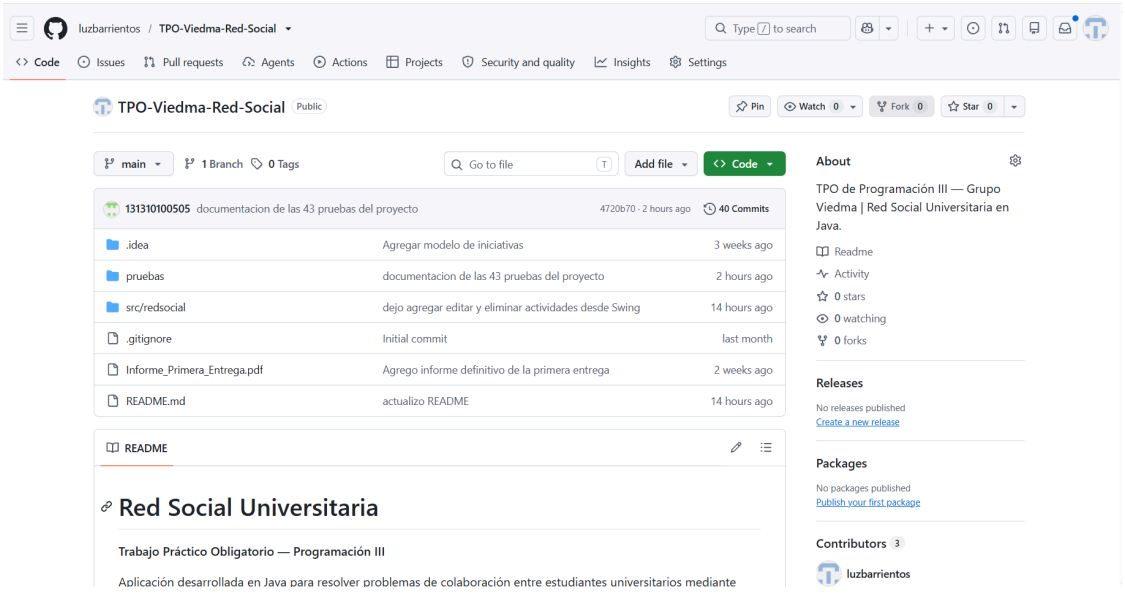


Figura 13. Repositorio compartido y estructura del proyecto.

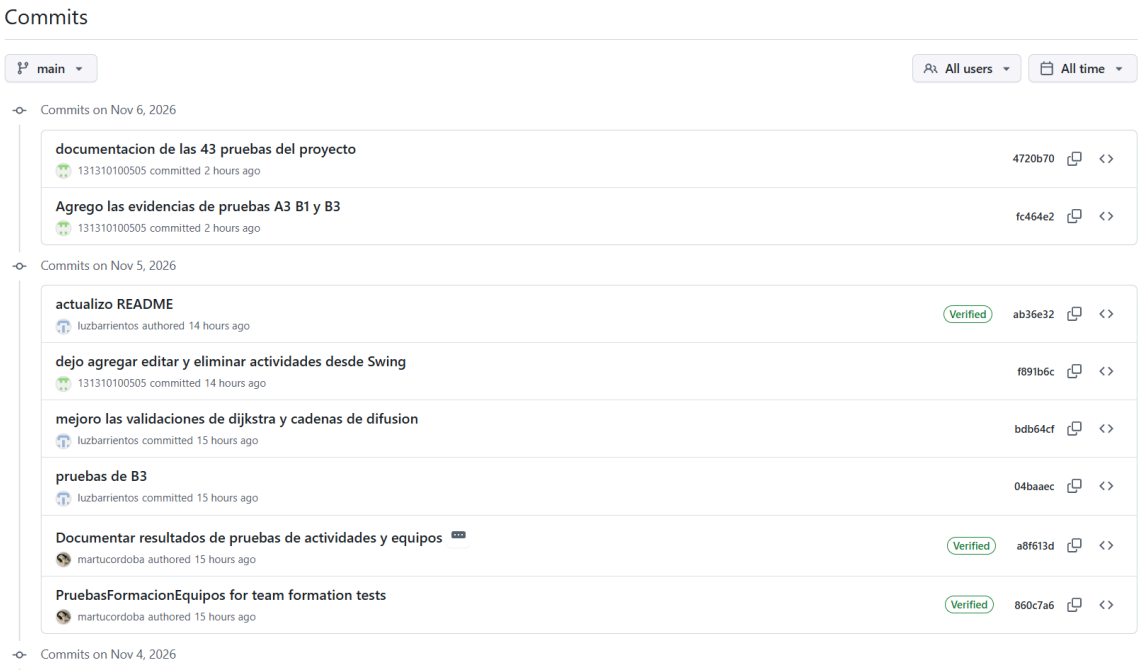


Figura 14. Historial de cambios registrados durante el desarrollo.

13. DIFICULTADES Y MEJORAS

Durante el desarrollo se presentaron decisiones relacionadas con la selección de algoritmos, el tratamiento de restricciones y la integración de funcionalidades.

Una dificultad que tuvimos fue distinguir problemas de optimización de problemas de enumeración.

Por ejemplo, Mochila 0/1 y A3 requieren maximizar un valor acumulado, mientras que B1 y B3 requieren encontrar todas las soluciones válidas.

También fue necesario incorporar validaciones para controlar entradas incorrectas y evitar resultados inconsistentes.

La integración con Swing fue una mejora respecto de la primera entrega, porque permitió utilizar los algoritmos desde una interfaz gráfica sin modificar su responsabilidad principal.

En particular, la pestaña Actividades incorporó operaciones para agregar, editar y eliminar elementos, además de restaurar los ejemplos.

Como posibles mejoras futuras se consideran la persistencia en bases de datos, la visualización gráfica de redes y las pruebas de rendimiento con entradas de mayor tamaño.

Estas ampliaciones no forman parte del alcance algorítmico principal del trabajo.

14. CONCLUSIÓN

El Trabajo Práctico Obligatorio de Programación III permitió aplicar diferentes estrategias de diseño y análisis de algoritmos a problemas de una red social universitaria.

Se implementaron seis funcionalidades utilizando Divide y Conquista, Greedy, Programación Dinámica y Backtracking.

Para cada problema se analizó su objetivo, se seleccionó una estrategia adecuada, se consideraron alternativas y se estudió su complejidad computacional.

La organización modular del código permitió separar los algoritmos, modelos, datos, pruebas e interfaz gráfica.

La incorporación de Swing facilitó la ejecución y visualización de las seis funcionalidades desde una aplicación de escritorio.

En la verificación final se ejecutaron 43 casos de prueba automatizados, obteniendo 43 resultados correctos y ninguna falla.

Además, las seis funcionalidades fueron comprobadas manualmente con los datos de ejemplo, obteniendo los resultados esperados.

El uso de Git y GitHub permitió organizar el trabajo colaborativo y conservar el historial de modificaciones.

En conclusión, el proyecto integró los contenidos teóricos de la materia con una implementación práctica en Java, destacando la importancia de elegir algoritmos apropiados, analizar su eficiencia y verificar sus resultados.

15. REFERENCIAS

Consigna: Trabajo Práctico Obligatorio de Programación III. Propuesta 1 — Red Social Universitaria para la Colaboración.

Informe de primera entrega: TPO Programación III — Grupo Viedma.

Repositorio del proyecto:
https://github.com/luzbarrientos/TPO-Viedma-Red-Social

Documentación de pruebas:
pruebas/CASOS_DE_PRUEBA

Evidencias: capturas de las pruebas automatizadas y de las seis funcionalidades ejecutadas mediante Java Swing.